

Natural-Language Agent Harnesses

Linyue Pan¹ Lexiao Zou² Shuo Guo¹ Jingchen Ni¹ Hai-Tao Zheng^{1*}

¹Shenzhen International Graduate School, Tsinghua University

²Harbin Institute of Technology (Shenzhen)

ply24@mails.tsinghua.edu.cn

zheng.haitao@sz.tsinghua.edu.cn

Abstract

Agent performance increasingly depends on *harness engineering*, yet harness design is usually buried in controller code and runtime-specific conventions, making it hard to transfer, compare, and study as a scientific object. We ask whether the high-level control logic of an agent harness can instead be externalized as a portable executable artifact. We introduce **Natural-Language Agent Harnesses** (NLAHs), which express harness behavior in editable natural language, and **Intelligent Harness Runtime** (IHR), a shared runtime that executes these harnesses through explicit contracts, durable artifacts, and lightweight adapters. Across coding and computer-use benchmarks, we conduct controlled evaluations of operational viability, module ablation, and code-to-text harness migration.

1 Introduction

Modern agents increasingly succeed or fail because of the surrounding *harness*: the control stack that structures multi-step reasoning, tool use, memory, delegation, and stopping beyond any single model call. A large body of research shows that externalized control patterns can be decisive, including reason-act loops (Yao et al., 2023), retrieval-augmented generation (Lewis et al., 2021), and explicit self-feedback (Shinn et al., 2023). Recent work has expanded this space toward explicit memory and self-evolution (Zhang et al., 2026), workflow generation (Li et al., 2024; Zheng et al., 2025), multi-agent orchestration (Fourney et al., 2024; Wang et al., 2025b; Ke et al., 2026; Costa, 2026; Xia et al., 2026), and interface-level test-time scaling and native tool execution (Muennighoff et al., 2025; Wang et al., 2024b; HKUDS, 2026). In parallel, long-context and long-horizon settings have exposed that the control stack—including state management, context curation, and context

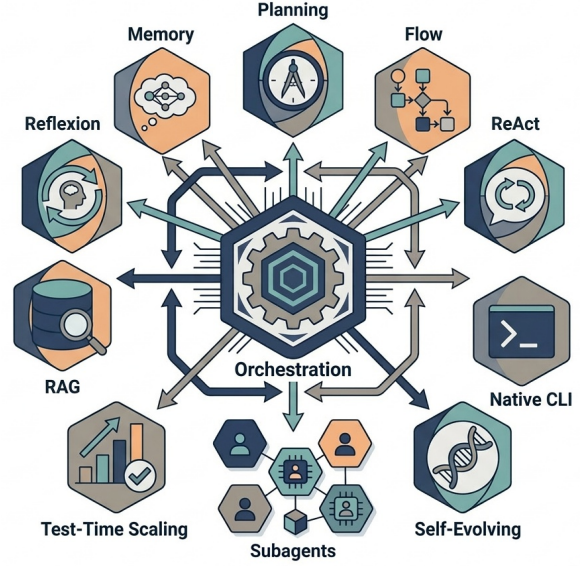


Figure 1: Examples of harness design patterns used by modern agents (reason-act, retrieval, reflection, verification, memory, search, orchestration).

folding—can bottleneck performance even when the base model is fixed (Liu et al., 2024; Chroma Research, 2025; Tang et al., 2025, 2026a,b; Sun et al., 2025; Su et al., 2026). The same pressure appears in scaffold-aware evaluation and increasingly demanding reasoning settings, where differences in scaffolds and harnesses can dominate outcomes even under fixed base models (Ding et al., 2026; An et al., 2025; Zhan et al., 2026b,a).

This shift reframes “prompt engineering” into the broader practice of *context engineering*: deciding what instructions, evidence, intermediate artifacts, and state should be made available at each step of a long run. Practitioner accounts emphasize that as tasks span many context windows, robust progress depends less on one-shot phrasing and more on durable state surfaces, validation gates, and clear responsibility boundaries (Anthropic, 2024, 2025a,b; Bui, 2026). In the same spirit, recent discussions of *harness engineering*

*Corresponding author.

treat the harness as a first-class systems object, not a thin wrapper around a model (OpenAI, 2026a; LangChain, 2026a,b, 2025).

Problem. Despite the growing importance of harness design, harness logic is rarely exposed as a coherent, portable artifact. In most agent systems, the effective harness is scattered across controller code, hidden framework defaults, tool adapters, verifier scripts, and runtime-specific assumptions (Lou et al., 2026; Shi et al., 2025; Chivukula et al., 2025; Wang et al., 2025a; Zhang et al., 2025). As a result, harnesses are difficult to transfer across runtimes, hard to compare fairly, and hard to ablate cleanly: two systems that nominally differ by one design choice often differ simultaneously in prompts, tool mediation, artifact conventions, verification gates, and state semantics (Liang et al., 2025; Cheng et al., 2025). This collapses evaluation into controller-bundle comparisons rather than module-level evidence.

Motivation. Natural-language artifacts such as AGENTS.md and skill bundles show that practical systems can package repository-local conventions and reusable procedures in portable text (AGENTS.md, 2026; AgentSkills, 2026). Recent work further treats these artifacts as learnable and benchmarkable objects through experience-driven skill creation, context-engineering skill evolution, reusable procedural memory, and cross-task skill evaluation (Hao et al., 2026; Ye et al., 2026; Mi et al., 2026; Zhang et al., 2026; Li et al., 2026b). What they establish, however, is feasibility at the level of reusable control knowledge, not an explicit executable harness representation. They typically attach local instructions or reusable routines, but they do not make harness-wide contracts, role boundaries, state semantics, failure handling, and runtime-facing adapters first-class and jointly executable under a shared runtime. This gap motivates our setting rather than closing it: we lift natural language from a carrier of reusable procedures to an explicit, executable harness object.

Thesis and approach. We ask whether the *design-pattern layer* inside agent harnesses can be made explicit as an *executable* natural-language object under shared runtime assumptions. We propose: (i) **Natural-Language Agent Harnesses** (NLAHs), a structured natural-language representation of harness control bound to explicit contracts and artifact carriers; and (ii) an **Intelligent Har-**

ness Runtime (IHR), which interprets NLAHs directly and separates shared *runtime charter* from task-family *harness logic*.

Contributions.

- **Formulation.** We formalize the harness *design-pattern layer* as an explicit representation object distinct from runtime policy and low-level execution hooks.
- **Representation ingredients.** We specify the components a natural-language harness must expose to be executable: contracts, roles, stage structure, adapters, scripts, state semantics, and a failure taxonomy.
- **Shared intelligent runtime.** We introduce Intelligent Harness Runtime (IHR), an in-loop LLM runtime that interprets harness logic directly while cleanly separating the runtime charter from harness logic.
- **Controlled evidence.** We conduct controlled experiments on shared-runtime behavioral effect (RQ1), module composition/ablation (RQ2), and paired code-to-text migration fidelity (RQ3) on coding and computer-use benchmarks.

2 Methodology

2.1 Harnesses and the pattern layer

We use *harness* to denote the orchestration layer that governs multiple model or agent calls for a task family. A harness specifies (i) **control**: how work is decomposed and scheduled; (ii) **contracts**: what artifacts must be produced, what gates must be satisfied, and when the run should stop; and (iii) **state**: what must persist across steps, branches, and delegated workers. By *context engineering* we mean designing the immediate prompt and retrieved context for a single call; a harness subsumes this, but also manages multi-step structure, tool mediation, verification, and durable state (Anthropic, 2025a,b).

The boundary between harness and runtime is analytical rather than absolute. In practice, some generic services (tool adapters, sandboxing, child lifecycle) may live in the runtime, while task-family policy (stages, artifact contracts, verifiers) lives in the harness. We make this boundary explicit for study: our goal is to compare, migrate, and ablate harness *pattern logic* under shared runtime assumptions.

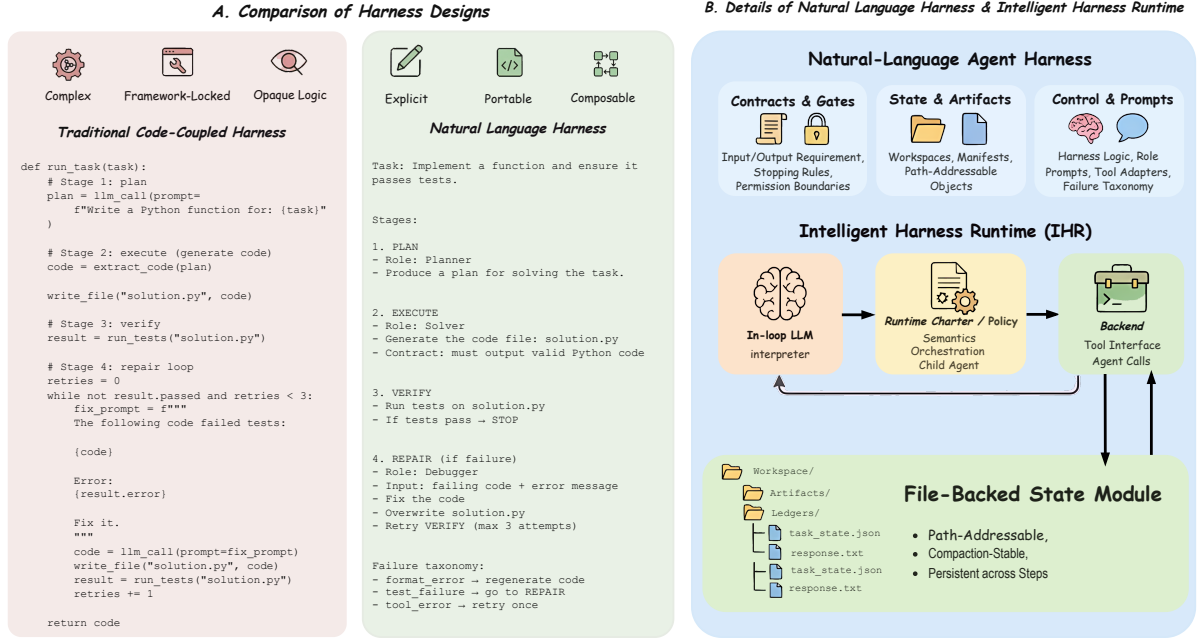


Figure 2: **Framework overview.** Intelligent Harness Runtime (IHR), with an in-loop LLM, a backend with tool access and child-agent support, and a runtime charter that specifies policy and semantics, executes a Natural-Language Agent Harness (NLAH), which exposes harness logic, roles, contracts, adapters, and state conventions, over task instances.

2.2 Intelligent Harness Runtime

Because NLAHs are written in natural language, executing them requires interpretation. IHR therefore places an LLM *inside* the runtime loop: at each step it reads (i) the harness, (ii) current state and environment, and (iii) the runtime charter, and then selects the next action consistent with contracts and budgets.

We decompose IHR into three components (Figure 2): (1) an **in-loop LLM** that interprets harness logic; (2) a **backend** that provides terminal tools and a first-class multi-agent interface (e.g., spawning and supervising child agents, ingesting returned artifacts); and (3) a **runtime charter** that defines the semantics of contracts, state, orchestration, and child lifecycle. In our experiments, child management uses the backend’s multi-agent tool surface (e.g., `spawn_agent`, `wait_agent`) (OpenAI, 2026c).

From model calls to agent calls. We lift a single completion into an *agent call* bounded by an explicit execution contract: required outputs, budgets, permission scope, completion conditions, and designated output paths. Appendix A gives the contract-based formalization used by the runtime.

2.3 Natural-Language Agent Harnesses

An NLAH is a structured natural-language representation of harness control intended to be executed by IHR. Natural language does not replace low-level deterministic code. Instead, it carries *editable, inspectable orchestration logic*, while adapters and scripts provide deterministic hooks (tests, linters, scrapers, verifiers).

Our formulation makes the following core components explicit:

- **Contracts:** required inputs and outputs, format constraints, validation gates, permission boundaries, **retry** and **stop** rules.
- **Roles:** role prompts (solver, verifier, researcher, orchestrator) with non-overlapping responsibilities.
- **Stage structure:** an explicit workload topology (e.g., plan → execute → verify → repair).
- **Adapters and scripts:** named hooks for deterministic actions (tests, verifiers, retrieval, parsing).
- **State semantics:** what persists across steps (artifacts, ledgers, child workspaces) and how it is reopened (paths, manifests).

- **Failure taxonomy:** named failure modes that drive recovery (missing artifact, wrong path, verifier failure, tool error, timeout).

2.4 File-backed state as an explicit module

Long-horizon autonomy fails in practice when critical state remains implicit or ephemeral. Recent context-folding work similarly treats explicit context management as essential, compressing completed sub-trajectories or dialogue history into reusable summaries and logs (Sun et al., 2025; Su et al., 2026). We therefore study an optional *file-backed state* module that externalizes durable state into path-addressable artifacts, improving stability under context truncation and branching (Anthropic, 2025b; Liu et al., 2024; Chroma Research, 2025).

Operationally, the module enforces three properties: **externalized** (state is written to artifacts rather than held only in transient context), **path-addressable** (later stages reopen the exact object by path), and **compaction-stable** (state survives truncation, restart, and delegation). Appendix B gives a canonical workspace and file-role mapping used in our experiments.

3 Experimental Design

3.1 Research questions

We evaluate whether harness pattern logic can become an executable and analyzable object under shared runtime assumptions.

- **RQ1 (Behavioral Effect).** Under fixed budgets, how do the shared runtime charter and benchmark-specific harness logic change agent behavior and task outcomes?
- **RQ2 (Composability).** Once patterns are explicit, can modules be composed and ablated at the pattern level?
- **RQ3 (Migration).** What differences remain between native code harnesses and reconstructed natural-language harnesses under a shared runtime?

3.2 Instantiation

In our instantiation, the backend is realized by Codex with terminal tools and a multi-agent interface; the shared runtime charter is carried by a fixed runtime skill; and benchmark-specific harness logic is carried by harness skills (OpenAI, 2025, 2026b). This factorization allows controlled ablations of

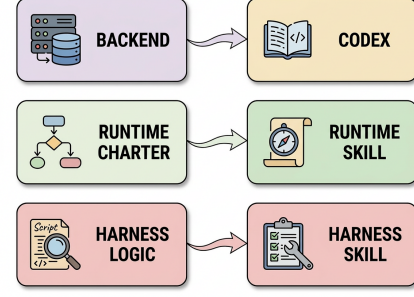


Figure 3: Realization mapping: backend + runtime skill (charter) + harness skill (task-family logic).

shared runtime policy versus benchmark-specific harness logic. Appendix C summarizes the shared runtime skill used in all IHR runs.

3.3 Benchmarks and harness families

We evaluate on two representative benchmark families that require multi-step control, tool use, durable state accumulation, and verification or evidence management.

Coding. SWE-bench Verified evaluates repository-grounded issue resolution; the main metric is issue resolution rate (Jimenez et al., 2024; Chowdhury et al., 2024). We study coding harness families including TRAE-style multi-candidate search (Team et al., 2025) and Live-SWE-Agent (Xia et al., 2025).

Computer use. OSWorld evaluates computer-use behavior grounded in real desktop environments; the main metric is task success rate (Xie et al., 2024). We study OS-Symphony as a holistic harness for computer-use agents (Yang et al., 2026).

3.4 Experimental setup

All experiments use the same IHR instantiation: Codex CLI version 0.114.0, model GPT-5.4 (OpenAI, 2026b), and reasoning effort xhigh. Runs execute on Ubuntu 24.04 servers with 64 CPU cores and 251 GiB of memory. To improve reproducibility and sandbox safety, all runs are executed in Docker containers. Per-task container caps are 32 vCPUs, 84 GiB memory, and 40 GiB storage.

Due to budget limits, the current paper reports results on benchmark subsets sampled once with a fixed random seed rather than on the full benchmark suites. The current subsets contain 125 SWE-bench Verified samples and 36 OSWorld samples.

Table 1: **RQ1: Outcome and process metrics under Full IHR and ablations.** The runtime skill carries shared charter; the harness skill carries benchmark-specific harness logic. Here, *w/o RTS* and *w/o HS* denote removing the runtime skill and harness skill, respectively.

Benchmark	Harness	Setting	Perf.	Prompt Tokens	Completion Tokens	Tool Calls	LLM Calls	Runtime (min)
SWE Verified	TRAE	Full IHR	74.4	16.3M	211k	642.6	414.3	32.5
		w/o RTS	76.0	11.1M	137k	451.9	260.5	16.6
		w/o HS	75.2	1.2M	13.6k	51.1	34.0	6.7
	Live-SWE	Full IHR	72.8	1.4M	17.0k	58.4	41.4	7.6
		w/o RTS	76.0	1.1M	11.7k	41.0	28.2	5.5
		w/o HS	75.2	1.2M	13.6k	51.1	34.0	6.7

Table 2: **RQ1 paired flips on SWE-bench Verified.** Counts compare Full IHR against each ablation on the same 125 stitched samples. F means only Full resolves, A means only the ablation resolves, and S means both settings agree.

Harness	vs. w/o RTS			vs. w/o HS		
	F	A	S	F	A	S
TRAE	4	6	115	7	8	110
Live-SWE	4	8	113	4	7	114

We plan to rerun the full benchmarks with GPT-5.4-mini and update the reported results in a future revision.

4 Results

4.1 RQ1: Behavioral effect

RQ1 tests whether the shared runtime charter and benchmark-specific harness logic materially change agent behavior and task outcomes under fixed budgets. The first result is that process metrics move much more than resolved rate. On SWE-bench Verified, the TRAE and Live-SWE rows stay within a narrow performance band, but Full IHR produces much larger changes in tokens, calls, and runtime than either ablation. RQ1 should therefore be read first as evidence that the shared runtime and harness logic change system behavior, not as a monotonic gain story.

The trajectory-level evidence shows that Full IHR is not a prompt wrapper. For TRAE, Full IHR sharply increases tool calls, LLM calls, and runtime, and Table 4 shows that about 90% of prompt tokens, completion tokens, tool calls, and LLM calls occur in delegated child agents rather than in the runtime-owned parent thread. The added budget therefore reflects multi-stage exploration,

candidate comparison, artifact handoff, and extra verification. Live-SWE is the lighter regime of the same mechanism: it raises process cost more moderately, but it still pushes the run toward a more explicit staged workflow than either ablation. Taken together, the runtime charter plus harness logic are behaviorally real controls rather than prompt decoration.

The next result is that most SWE instances do not flip. Across both TRAE and Live-SWE, more than 110 of 125 stitched SWE samples agree between Full IHR and each ablation (Table 2). The meaningful differences are therefore concentrated in a small frontier of component-sensitive cases. Full IHR behaves more like a solved-set replacer than a uniform frontier expander: it creates some Full-only wins, but it also loses some direct-path repairs that lighter settings retain. Appendix D summarizes representative component-sensitive SWE cases.

The most informative failures are alignment failures rather than random misses. On `matplotlib__matplotlib-24570`, TRAE Full expands into a large candidate search, runs multiple selector and revalidation stages, and still ends with a locally plausible patch that misses the official evaluator. Live-SWE exposes the lighter analogue on cases such as `django__django-14404`, `sympy__sympy-23950`, and `django__django-13406`, where extra structure makes the run more organized and more expensive while drifting away from the shortest benchmark-aligned repair path or from the evaluator’s final acceptance object. These failures matter because they show not that the harness is inert, but that it can reshape local success signals in ways that do not always align with benchmark acceptance.

Table 3: **RQ2: Module composition and ablation.** Within each benchmark, we begin from a benchmark-specific *Basic* starting point and add one module at a time.

Benchmark	Basic	File-Backed State	Evidence-Backed Answering	Verifier	Self-Evolution	Multi-Candidate Search	Dynamic Orchestration
SWE Verified	75.2	76.8 _{+1.6}	76.8 _{+1.6}	74.4 _{-0.8}	80.0 _{+4.8}	72.8 _{-2.4}	75.2 _{0.0}
OSWorld	41.7	47.2 _{+5.5}	41.7 _{0.0}	33.3 _{-8.4}	44.4 _{+2.7}	36.1 _{-5.6}	44.4 _{+2.7}

Table 4: **TRAE NLAH usage split.** Approximate share of total usage attributable to the runtime-owned parent thread vs. delegated child agents (per-sample averages).

Metric	Runtime-owned parent	Delegated child agents
Prompt tokens	8.5%	91.5%
Completion tokens	8.1%	91.9%
Tool calls	9.8%	90.2%
LLM calls	9.4%	90.6%

4.2 RQ2: Harness pattern ablations

RQ2 asks whether, once harness patterns are made explicit, they can be composed and ablated as modules under a shared substrate.

For clarity, *Basic* is benchmark-specific in this table. On SWE, Basic is a bare Codex baseline with shell plus file reading, writing, and editing tools. On OSWorld, Basic is the NLAH realization of OS-Symphony before adding the extra RQ2 modules. We then add one module at a time: file-backed state, evidence-backed answering, a verifier stage, self-evolution, multi-candidate search, and dynamic orchestration. This makes the SWE rows close to tool-and-workflow ablations over a minimal coding agent, whereas the OSWorld rows are ablations over an already structured computer-use harness.

The first pattern is that module effects concentrate on a small solved frontier rather than shifting the whole benchmark uniformly. Most tasks are either solved robustly by nearly all conditions or remain unsolved across conditions, so the informative differences come from boundary cases that flip under changed control logic. RQ2 should therefore be read as a study of how modules reshape the frontier of difficult cases, not just as a ranking over mean scores.

The second pattern is that the modules fall into two qualitatively different families. Self-evolution is the clearest example of a module that improves

the solve loop itself. The trajectory evidence suggests that its main benefit is not open-ended reflection, but a more disciplined acceptance-gated attempt loop that keeps the search narrow until failure signals justify another pass. Cases such as `scikit-learn__scikit-learn-25747` fit this interpretation: the module succeeds by forcing a cleaner success criterion around an ordinary repair attempt, not by expanding into an expensive tree of candidates. By contrast, file-backed state and evidence-backed answering mainly improve process structure. They leave durable external signatures such as task histories, manifests, and analysis sidecars, which is strong evidence that they really externalize state and evidence handling. Their gains remain mild, which suggests that they improve auditability, handoff discipline, and trace quality more directly than semantic repair ability.

The third pattern is that more explicit structure does not automatically mean better end-task performance. Dynamic orchestration is behaviorally real rather than inert because it changes which SWE instances are solved, but it mostly acts as a solved-set replacer instead of expanding the frontier. Verifier and multi-candidate search show a harsher version of the same principle. Verifier adds a genuine independent checking layer, yet failures such as `sympy__sympy-23950` show that verifier-level acceptance can still diverge from benchmark-level acceptance. Multi-candidate search makes search behavior more visible, but under the current runtime and budget it appears too overhead-heavy and infrastructure-sensitive to convert that richer behavior into better aggregate outcomes.

OSWorld points in the same direction from a different starting point: because its Basic condition is already a structured harness, the most useful additions are again the lighter modules that tighten local organization without adding a heavy extra acceptance layer. Overall, RQ2 does not support a simple “more structure is always better” story. The stronger interpretation is that explicit modules help when they tighten the path from intermediate be-

Table 5: **RQ3: Paired code-to-text harness comparison.** Each harness is evaluated as original source code vs. reconstructed NLAH under IHR. Here, *Code* denotes the original source implementation.

Benchmark	Harness	Realization	Perf.	Prompt Tokens	Completion Tokens	Agent Calls	Tool Calls	LLM Calls	Runtime (min)
OSWorld	OS-Symphony	Code	30.4	11.4M	147.2k	99	651	1.2k	361.5
		NLAH	47.2	15.7M	228.5k	72	683	34	140.8

havior to the evaluator’s acceptance condition, and help less when they mainly add local process layers whose notion of success is only weakly aligned with the final benchmark. Appendix E adds token-cost and Basic-union views together with representative case studies that make the same mechanism-level pattern more concrete.

4.3 RQ3: Code-to-text harness migration

RQ3 is a paired migration study: each harness appears in two realizations (source code vs. reconstructed NLAH), evaluated under a shared reporting schema (Table 5). The target is task-level equivalence—comparable exposed logic, contracts, and benchmark-facing artifacts—not identical internal traces. On OSWorld, the migrated OS-Symphony realization reaches 47.2 versus 30.4 for the native code harness. The more important difference, however, is behavioral rather than purely numerical. Native OS-Symphony externalizes control as a screenshot-grounded repair loop: verify the previous step, inspect the current screen, choose the next GUI action, and retry locally when focus or selection errors occur. Under IHR, the same task family tends to re-center around file-backed state and artifact-backed verification. Runs materialize task files, ledgers, and explicit artifacts, and they switch more readily from brittle GUI repair to file, shell, or package-level operations when those operations provide a stronger completion certificate.

The retained RQ3 archives make this relocation concrete. The native side exposes 36 main traces plus 7 short nested `search_1` traces, whereas the migrated side exposes 34 retained inner event streams and 2 missing-inner-stream stubs. This means the native topology is a desktop control loop with occasional detachable tutorial detours, while the migrated topology is a contract-first runtime flow whose state lives in task files, ledgers, and artifacts.

Search is preserved functionally, but relocated topologically. Among the 6 native-search samples whose migrated inner streams are retained, only 3 also contain explicit `web_search`, and 1 additional

migrated sample uses `web_search` without a native `search_1` branch. Search therefore survives less as an auxiliary sub-agent branch and more as in-band runtime support for substrate choice and deterministic repair.

Verification shifts even more strongly. Native traces often stop on screen plausibility, whereas migrated runs more often close on path-addressable evidence such as a written file, a reopened document, a package-level object, or a system query. This shift matters because OSWorld tasks often fail not at first-pass intent, but at recovery and closure.

Retained migrated traces are also denser, but that density should not be read as a raw action multiplier. Across paired retained samples, native main traces average 18.1 steps, while migrated traces average about 18.2 unique command starts but 58.5 total logged events because the runtime also preserves started/completed pairs, bookkeeping, and explicit artifact handling. The extra density is therefore better interpreted as observability plus recovery scaffolding than as dramatically more task actions. These tendencies are consistent with the OSWorld module results in RQ2, where file-backed state is the strongest positive addition, and they help explain why the NLAH realization obtains a modest performance gain rather than a penalty.

Case sketches. Representative cases make the same mechanism concrete. In a system-configuration task, the native run stays trapped in GUI focus repair, whereas the NLAH realization shifts to shell-side configuration and closes only after explicit `sshd` validation. In a spreadsheet task, the native run reaches apparent visual progress yet fails to close robustly, whereas the migrated harness writes the target artifact deterministically and reopens it before completion. In a presentation task, the native harness can retrieve the right tutorial path yet still struggle with object binding and drag control, whereas the migrated harness edits the `.pptx` package directly and verifies the resulting slide artifact. Taken together, these cases suggest that the main migration effect is not loss of

high-level orchestration, but relocation of reliability mechanisms from local screen repair to durable runtime state and artifact-backed closure.

5 Discussion

Code versus natural language. We do not argue that natural language should replace code. Instead, natural language carries editable high-level harness logic, while code remains responsible for deterministic operations, tool interfaces, and sandbox enforcement. The scientific claim is about the unit of comparison: externalizing harness pattern logic as a readable, executable object under shared runtime semantics.

Why natural language still matters. A natural concern is whether stronger foundation models reduce the value of natural-language control. Empirically, gains from complex prompt engineering can diminish or become brittle in some settings (Wang et al., 2024a; Cao et al., 2024). However, our results support a different interpretation for agent systems: natural language remains important when used to specify *harness-level control*—roles, contracts, verification gates, durable state semantics, and delegation boundaries—rather than only one-shot prompt phrasing. This framing is consistent with practitioner accounts that emphasize context engineering and long-running harness design (Anthropic, 2025a,b; OpenAI, 2026a; LangChain, 2026a). It is also compatible with emerging scaffold-aware evaluation and harness-synthesis research that treat the surrounding control stack as part of the system under evaluation (Ding et al., 2026; Lou et al., 2026; Chen et al., 2026b).

Searching harness representations. Once harnesses are explicit objects, they become a search space. Explicit harness modules can be manually designed, retrieved, migrated, recombined, and systematically ablated under shared assumptions. Longer term, this suggests automated search and optimization over harness representations rather than opaque bundle engineering, enabling harness engineering to become a more controlled scientific object.

6 Related Work

Prompts as programs and LLM programming systems. Several lines of work treat prompts and LLM calls as programmable objects. Liang et al. argue that some prompts are programs and study

how developers engineer prompt-enabled software systems (Liang et al., 2025). Promptware engineering further frames prompt-enabled systems as a software-engineering object with concerns of maintainability, testing, and integration (Chen et al., 2026b). At the language and systems level, LMQL adds constraints and control flow to prompting (Beurer-Kellner et al., 2023), DSPy compiles declarative LM pipelines (Khattab et al., 2024), APPL integrates prompts and Python programs (Dong et al., 2025), and SGLang provides an execution system for structured language-model programs (Zheng et al., 2024). Cheng et al. study mechanisms for sharing state between prompts and programs (Cheng et al., 2025). These works primarily program calls or pipelines; our focus is the harness layer that governs multi-step *agent calls*, artifact contracts, delegation, verification, and durable state.

Agent control patterns and orchestration.

Core agent control patterns include reason-act loops (Yao et al., 2023), retrieval augmentation (Lewis et al., 2021), and reflection/self-feedback (Shinn et al., 2023). Subsequent work expands this space toward memory and self-evolution (Zhang et al., 2026; Xia et al., 2025), multi-agent generalists (Fourney et al., 2024), workflow generation (Li et al., 2024; Zheng et al., 2025), and dynamic topology/routing (Wang et al., 2025b,c; Yue et al., 2025; Ke et al., 2026; Costa, 2026). Our work is complementary: we do not propose a new orchestration algorithm, but instead externalize the harness pattern logic as an executable representation under a shared runtime.

Natural language to workflows, constraints, and enforcement.

Several systems translate natural language into workflows or executable constraints. AutoFlow generates workflows from natural-language descriptions (Li et al., 2024), FlowAgent studies compliance vs. flexibility (Shi et al., 2025), and Agint compiles software-engineering agents into agentic graphs (Chivukula et al., 2025). AgentSpec focuses on runtime enforcement mechanisms (Wang et al., 2025a), and ContextCov derives executable constraints from agent instruction files (Sharma, 2026). OpenProse and Lobster expose workflow/specification systems close to natural-language authoring (OpenProse, 2026; OpenClaw, 2026). In contrast to compiling to a runtime-owned IR, IHR interprets harness logic directly, relying on explicit contracts and durable

artifacts for auditability.

Harness engineering in practice and automatic harness synthesis. Recent context-folding work tackles a nearby systems problem by compressing long interaction histories for long-horizon agents (Sun et al., 2025; Su et al., 2026). Recent public engineering accounts describe harness engineering as a primary driver of robustness in long-running agents (Anthropic, 2024, 2025a,b,c, 2026b,a; OpenAI, 2026a; LangChain, 2026b,a; Bui, 2026). On the research side, AutoHarness explicitly treats harness synthesis as an optimization target, automatically producing code harnesses that improve agent behavior (Lou et al., 2026). General Modular Harness studies modular harness structure in multi-turn environments (Zhang et al., 2025). Our work differs by focusing on the harness *design-pattern layer* as a natural-language representation object that can be executed under a shared intelligent runtime.

Reusable instruction carriers and skills. Natural-language carriers such as AGENTS.md, AgentSkills, and related skill bundles demonstrate that portable, attachable operational knowledge can be packaged as text and reused across environments (AGENTS.md, 2026; AgentSkills, 2026). Recent skill work pushes this further by treating skills as objects that can be created from experience, evolved for context engineering, or maintained as reusable procedural memory rather than fixed one-off prompts (Hao et al., 2026; Ye et al., 2026; Mi et al., 2026; Zhang et al., 2026). Skills also provide an alternative modularity substrate: a single agent equipped with a skill library can sometimes replace explicit multi-agent communication, although this substitution breaks when tasks require genuine parallelism, private state, or adversarial role structure (Li, 2026). At the ecosystem level, AgentSkillOS studies organizing and orchestrating large skill collections, while SkillsBench, SkillCraft, and PinchBench evaluate cross-task transfer, higher-level tool composition, and practical skill invocation under diverse tasks (Li et al., 2026a,b; Chen et al., 2026a; PinchBench, 2026). We extend this idea from reusable local guidance to executable harness-level control.

7 Conclusion

We study whether the harness design-pattern layer can be externalized as an executable, compara-

ble, and ablatable object. We propose Natural-Language Agent Harnesses and an Intelligent Harness Runtime that interprets harness logic directly under shared runtime semantics. Across the current coding and computer-use benchmarks, we provide controlled evidence that this stack is operationally viable, enables module-level composition and ablation, and supports meaningful code-to-text harness migration studies. These results suggest a path toward harness representation science, where harness modules become first-class research artifacts rather than incidental glue around models.

Limitations

Natural language is less precise than code, and some harness mechanisms cannot be recovered faithfully from text, especially when they rely on hidden service-side state, proprietary schedulers, or training-induced behaviors not observable from released artifacts. Runtime contamination remains a real risk: a strong shared runtime charter may absorb part of the behavior that one might otherwise attribute to harness text. Module-level ablation is not strict causal identification; textual representations can introduce confounds such as instruction salience and prompt length.

Broader impact and risks

Externalizing harness modules can reduce development cost, improve comparability, and encourage reuse of robust workflows. However, portable harness logic and scripts may also lower the barrier to spreading risky workflows. Because harnesses mediate tool use, artifact handling, and delegation, they can introduce new attack surfaces for prompt injection, malicious tool grafting, or supply-chain contamination. Deployments should combine provenance tracking, review, permission control, and sandbox isolation.

References

- AgentSkills. 2026. [Agentskills](#). Website home page. Accessed: 2026-03-13.
- AGENTS.md. 2026. [Agents.md](#). Community specification website. Accessed: 2026-03-13.
- Shengnan An, Xunliang Cai, Xuezhi Cao, Xiaoyu Li, Yehao Lin, Junlin Liu, Xinxuan Lv, Dan Ma, Xuanlin Wang, Ziwen Wang, and Shuang Zhou. 2025. [Amo-bench: Large language models still struggle in high school math competitions](#). *Preprint*, arXiv:2510.26768.

- Anthropic. 2024. [Building effective agents](#). Engineering blog. Published: 2024-12-19. Accessed: 2026-03-12.
- Anthropic. 2025a. [Effective context engineering for ai agents](#). Engineering blog. Published: 2025-09-29. Accessed: 2026-03-12.
- Anthropic. 2025b. [Effective harnesses for long-running agents](#). Engineering blog. Published: 2025-11-26. Accessed: 2026-03-12.
- Anthropic. 2025c. [How we built our multi-agent research system](#). Engineering blog. Published: 2025-06-13. Accessed: 2026-03-12.
- Anthropic. 2026a. [Claude code subagents](#). Documentation page. Accessed: 2026-03-06.
- Anthropic. 2026b. [How claude remembers your project](#). Documentation page. Accessed: 2026-03-12.
- Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. [Prompting is programming: A query language for large language models](#). *Proc. ACM Program. Lang.*, 7(PLDI).
- Nghi D. Q. Bui. 2026. [Building effective ai coding agents for the terminal: Scaffolding, harness, context engineering, and lessons learned](#). *Preprint*, arXiv:2603.05344.
- Bowen Cao, Deng Cai, Zhisong Zhang, Yuexian Zou, and Wai Lam. 2024. [On the worst prompt performance of large language models](#). *Preprint*, arXiv:2406.10248.
- Shiqi Chen, Jingze Gai, Ruochen Zhou, Jinghan Zhang, Tongyao Zhu, Junlong Li, Kangrui Wang, Zihan Wang, Zhengyu Chen, Klara Kaleb, Ning Miao, Siyang Gao, Cong Lu, Manling Li, Junxian He, and Yee Whye Teh. 2026a. [Skillcraft: Can llm agents learn to use tools skillfully?](#) *Preprint*, arXiv:2603.00718.
- Zhenpeng Chen, Chong Wang, Weisong Sun, Xuanzhe Liu, Jie M. Zhang, and Yang Liu. 2026b. [Promptware engineering: Software engineering for prompt-enabled systems](#). *Preprint*, arXiv:2503.02400.
- Ellie Y. Cheng, Logan Weber, Tian Jin, and Michael Carbin. 2025. [Sharing state between prompts and programs](#). *Preprint*, arXiv:2512.14805.
- Abhi Chivukula, Jay Somasundaram, and Vijay Somasundaram. 2025. [Agint: Agentic graph compilation for software engineering agents](#). *Preprint*, arXiv:2511.19635.
- Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubeh, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. 2024. [Introducing SWE-bench verified](#).
- Chroma Research. 2025. [Context rot: How increasing input tokens impacts llm performance](#). Research article. Accessed: 2026-03-06.
- Igor Costa. 2026. [Agentspaw: Adaptive multi-agent collaboration through dynamic spawning for long-horizon code generation](#). *Preprint*, arXiv:2602.07072.
- Deming Ding, Shichun Liu, Enhui Yang, Jiahang Lin, Ziyang Chen, Shihan Dou, Honglin Guo, Weiyu Cheng, Pengyu Zhao, Chengjun Xiao, Qunhong Zeng, Qi Zhang, Xuanjing Huang, Qidi Xu, and Tao Gui. 2026. [Octobench: Benchmarking scaffold-aware instruction following in repository-grounded agentic coding](#). *Preprint*, arXiv:2601.10343.
- Honghua Dong, Qidong Su, Yubo Gao, Zhaoyu Li, Yangjun Ruan, Gennady Pekhimenko, Chris J. Madison, and Xujie Si. 2025. [APPL: A prompt programming language for harmonious integration of programs and large language model prompts](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1243–1266, Vienna, Austria. Association for Computational Linguistics.
- Adam Fourney, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, Friederike Niedtner, Grace Proebsting, Griffin Bassman, Jack Gerrits, Jacob Alber, Peter Chang, Ricky Loynd, Robert West, Victor Dibia, Ahmed Awadallah, Ece Kamar, Rafah Hosn, and Saleema Amershi. 2024. [Magentic-one: A generalist multi-agent system for solving complex tasks](#). *Preprint*, arXiv:2411.04468.
- Zhezheng Hao, Hong Wang, Jian Luo, Jianqing Zhang, Yuyan Zhou, Qiang Lin, Can Wang, Hande Dong, and Jiawei Chen. 2026. [Recreate: Reasoning and creating domain agents driven by experience](#). *Preprint*, arXiv:2601.11100.
- HKUDS. 2026. [CLI-Anything: Making ALL Software Agent-Native](#). GitHub repository. Repository created: 2026-03-08. Accessed: 2026-03-23.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. [SWE-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations*.
- Zixuan Ke, Yifei Ming, Austin Xu, Ryan Chin, Xuan-Phi Nguyen, Prathyusha Jwalapuram, Jiayu Wang, Semih Yavuz, Caiming Xiong, and Shafiq Joty. 2026. [Mas-orchestra: Understanding and improving multi-agent reasoning through holistic orchestration and controlled benchmarks](#). *Preprint*, arXiv:2601.14652.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. [Dspy: Compiling declarative language model calls into state-of-the-art](#)

- pipelines. In *International Conference on Learning Representations*, volume 2024, pages 54928–54958.
- LangChain. 2025. [Deep agents](#). Engineering blog. Published: 2025-07-30. Accessed: 2026-03-12.
- LangChain. 2026a. [The anatomy of an agent harness](#). Engineering blog. Published: 2026-03-10. Accessed: 2026-03-12.
- LangChain. 2026b. [Improving deep agents with harness engineering](#). Engineering blog. Published: 2026-02-17. Accessed: 2026-03-12.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). *Preprint*, arXiv:2005.11401.
- Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026a. [Organizing, orchestrating, and benchmarking agent skills at ecosystem scale](#). *Preprint*, arXiv:2603.02176.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, and 21 others. 2026b. [Skillsbench: Benchmarking how well agent skills work across diverse tasks](#). *Preprint*, arXiv:2602.12670.
- Xiaoxiao Li. 2026. [When single-agent with skills replace multi-agent systems and when they fail](#). *Preprint*, arXiv:2601.04748.
- Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. [Autoflow: Automated workflow generation for large language model agents](#). *Preprint*, arXiv:2407.12821.
- Jenny T. Liang, Melissa Lin, Nikitha Rao, and Brad A. Myers. 2025. [Prompts are programs too! understanding how developers build software containing prompts](#). *Proc. ACM Softw. Eng.*, 2(FSE).
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. [Lost in the middle: How language models use long contexts](#). *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. 2026. [Autoharness: improving llm agents by automatically synthesizing a code harness](#). *Preprint*, arXiv:2603.03329.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. 2026. [Procmem: Learning reusable procedural memory from experience via non-parametric ppo for llm agents](#). *Preprint*, arXiv:2602.01869.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. [s1: Simple test-time scaling](#). *Preprint*, arXiv:2501.19393.
- OpenAI. 2025. [Introducing codex](#). Product announcement. Published: 2025-05-16. Accessed: 2026-03-13.
- OpenAI. 2026a. [Harness engineering: leveraging codex in an agent-first world](#). Engineering blog. Published: 2026-02-11. Accessed: 2026-03-13.
- OpenAI. 2026b. [Introducing gpt-5.4](#). Product announcement. Published: 2026-03-05. Accessed: 2026-03-13.
- OpenAI. 2026c. [Multi-agents](#). Documentation page. Accessed: 2026-03-10.
- OpenClaw. 2026. [Lobster](#). GitHub repository. First public repository commit: 2026-01-17; Accessed: 2026-03-11.
- OpenProse. 2026. [Openprose](#). GitHub repository. First public repository commit: 2026-01-03; Accessed: 2026-03-11.
- PinchBench. 2026. [Pinchbench](#). GitHub repository. Accessed: 2026-03-08.
- Reshabh K Sharma. 2026. [Contextcov: Deriving and enforcing executable constraints from agent instruction files](#). *Preprint*, arXiv:2603.00822.
- Yuchen Shi, Siqui Cai, Zihan Xu, Yuei Qin, Gang Li, Hang Shao, Jiawei Chen, Deqing Yang, Ke Li, and Xing Sun. 2025. [Flowagent: Achieving compliance and flexibility for workflow agents](#). *Preprint*, arXiv:2502.14345.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). *Preprint*, arXiv:2303.11366.
- Jin Su, Runnan Fang, Yeqiu Li, Xiaobin Wang, Shihao Cai, Pengjun Xie, Ningyu Zhang, and Fajie Yuan. 2026. [U-fold: Dynamic intent-aware context folding for user-centric agents](#). *Preprint*, arXiv:2601.18285.
- Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2025. [Scaling long-horizon llm agent via context-folding](#). *Preprint*, arXiv:2510.11967.
- Jiwei Tang, Shilei Liu, Zhicheng Zhang, Qingsong Lv, Runsong Zhao, Tingwei Lu, Langming Liu, Haibin Chen, Yujin Yuan, Hai-Tao Zheng, Wenbo Su, and Bo Zheng. 2026a. [Read as human: Compressing context via parallelizable close reading and skimming](#). *Preprint*, arXiv:2602.01840.

- Jiwei Tang, Jin Xu, Tingwei Lu, Zhicheng Zhang, Yiming Zhao, Lin Hai, and Hai-Tao Zheng. 2025. [Perception compressor: A training-free prompt compression framework in long context scenarios](#). *Preprint*, arXiv:2409.19272.
- Jiwei Tang, Zhicheng Zhang, Shunlong Wu, Jingheng Ye, Lichen Bai, Zitai Wang, Tingwei Lu, Lin Hai, Yiming Zhao, Hai-Tao Zheng, and Hong-Gee Kim. 2026b. [Gmsa: Enhancing context compression via group merging and layer semantic alignment](#). *Preprint*, arXiv:2505.12215.
- Trae Research Team, Pengfei Gao, Zhao Tian, Xiangxin Meng, Xincheng Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, Yun Lin, Yingfei Xiong, Chao Peng, and Xia Liu. 2025. [Trae agent: An llm-based agent for software engineering with test-time scaling](#). *Preprint*, arXiv:2507.23370.
- Guoqing Wang, Zeyu Sun, Zhihao Gong, Sixiang Ye, Yizhou Chen, Yifan Zhao, Qingyuan Liang, and Dan Hao. 2024a. [Do advanced language models eliminate the need for prompt engineering in software engineering?](#) *Preprint*, arXiv:2411.02093.
- Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025a. [Agentspec: Customizable runtime enforcement for safe and reliable llm agents](#). *Preprint*, arXiv:2503.18666.
- Song Wang, Zhen Tan, Zihan Chen, Shuang Zhou, Tianlong Chen, and Jundong Li. 2025b. [AnyMAC: Cascading flexible multi-agent collaboration via next-agent prediction](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11555–11567, Suzhou, China. Association for Computational Linguistics.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024b. [Executable code actions elicit better llm agents](#). *Preprint*, arXiv:2402.01030.
- Zhexuan Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. 2025c. [Agent-Dropout: Dynamic agent elimination for token-efficient and high-performance LLM-based multi-agent collaboration](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 24013–24035, Vienna, Austria. Association for Computational Linguistics.
- Chunqiu Steven Xia, Zhe Wang, Yan Yang, Yuxiang Wei, and Lingming Zhang. 2025. [Live-swe-agent: Can software engineering agents self-evolve on the fly?](#) *Preprint*, arXiv:2511.13646.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. [Skillrl: Evolving agents via recursive skill-augmented reinforcement learning](#). *Preprint*, arXiv:2602.08234.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. [Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 52040–52094. Curran Associates, Inc.
- Bowen Yang, Kaiming Jin, Zhenyu Wu, Zhaoyang Liu, Qiushi Sun, Zehao Li, JingJing Xie, Zhoumianze Liu, Fangzhi Xu, Kanzhi Cheng, Qingyun Li, Yian Wang, Yu Qiao, Zun Wang, and Zichen Ding. 2026. [Os-symphony: A holistic framework for robust and generalist computer-using agent](#). *Preprint*, arXiv:2601.07779.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). In *The Eleventh International Conference on Learning Representations*.
- Haoran Ye, Xuning He, Vincent Arak, Haonan Dong, and Guojie Song. 2026. [Meta context engineering via agentic skill evolution](#). *Preprint*, arXiv:2601.21557.
- Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyang Qi. 2025. [MasRouter: Learning to route LLMs for multi-agent systems](#). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, Vienna, Austria. Association for Computational Linguistics.
- Shaoxiong Zhan, Yanlin Lai, Zheng Liu, Hai Lin, Shen Li, Xiaodong Cai, Zijian Lin, Wen Huang, and Hai-Tao Zheng. 2026a. [3viewsense: Spatial and mental perspective reasoning from orthographic views in vision-language models](#). *Preprint*, arXiv:2603.07751.
- Shaoxiong Zhan, Yanlin Lai, Ziyu Lu, Dahua Lin, Ziqing Yang, and Fei Tan. 2026b. [Mathsmith: Towards extremely hard mathematical reasoning by forging synthetic problems with a reinforced policy](#). *Preprint*, arXiv:2508.05592.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. 2026. [Memskill: Learning and evolving memory skills for self-evolving agents](#). *Preprint*, arXiv:2602.02474.
- Yuxuan Zhang, Haoyang Yu, Lanxiang Hu, Haojian Jin, and Hao Zhang. 2025. [General modular harness for llm agents in multi-turn gaming environments](#). *Preprint*, arXiv:2507.11633.
- Chengqi Zheng, Jianda Chen, Yueming Lyu, Wen Zheng Terence Ng, Haopeng Zhang, Yew-Soon Ong, Ivor Tsang, and Haiyan Yin. 2025. [Mermaidflow: Redefining agentic workflow generation via safety-constrained evolutionary programming](#). *Preprint*, arXiv:2505.22967.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. Sglang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems, NIPS '24*, Red Hook, NY, USA. Curran Associates Inc.

A From model calls to agent calls

A multimodal LLM can be viewed as a mapping from context c to output y , where the context may include text, images, or video:

$$y = \text{LM}_m(c).$$

To support tool use, we assume a structured action format that can invoke external tools.

We lift a base model call into an *agent call*. We define a task as

$$T = (p, F_{\text{in}}, \kappa),$$

where p is the task prompt, F_{in} is the set of input files or linked resources, and κ is an execution contract (required outputs, budget, permission scope, completion conditions, designated output paths).

An agent call is

$$\text{AgentCall}(T, \Omega_t^{\text{in}}) = (A_t, \Delta\Omega_t, y_t),$$

where Ω_t^{in} is the visible environment and file state at call start, A_t is the designated artifact set, $\Delta\Omega_t$ are environment modifications, and y_t is a normalized final response pointing to artifacts and stating success or failure. A single model call is a degenerate special case where κ enforces one-shot answering with no external action.

B Canonical workspace for file-backed state

The file-backed module treats a canonical workspace as the authoritative carrier of durable cross-step state.

Canonical workspace	Abstract object	Example carrier	Role
run/ TASK.md	Task object	TASK.md	Run-local task statement, linked inputs, designated outputs
harness-skill/ SKILL.md references/ scripts/ state/ task_history.jsonl children/ 001/ TASK.md SKILL.md inputs/ scripts/ scratch/ RESPONSE.md artifacts/ RESPONSE.md artifacts/	Run-level final response	RESPONSE.md, children/*/RESPONSE.md	Normalized outcome together with success or failure status and artifact pointers
	Harness skill	harness-skill/SKILL.md, harness-skill/references/, harness-skill/scripts/	Task-family control logic together with reusable references and scripts
	Task history	state/ task_history.jsonl	Append-only record of child invocations and state promotions used to recover active runtime state
	Child workspace	children/*/TASK.md, children/*/SKILL.md, children/*/inputs/, children/*/scripts/, children/*/scratch/, children/*/artifacts/	Child-local task packet, copied inputs, tools, scratch space, and local artifacts
	Final artifacts	artifacts/	Benchmark-facing outputs and designated deliverables

Table 6: Canonical workspace layout and file-role mapping for file-backed state.

C Outline of the shared runtime skill

The fixed runtime skill used in IHR is not a benchmark-specific harness. It encodes the shared runtime charter that makes different harness skills executable under a common substrate. In operational terms, the charter enforces five ideas:

- **Runtime-only parent role.** The top-level agent is an orchestrator rather than the direct worker, so even a nominally single-agent harness is realized as “parent runtime + one task child.” This keeps substantive workspace work inside child agents and makes delegation boundaries inspectable.

Table 7: **Representative component-sensitive SWE cases for RQ1.**

Sample	Outcome pattern	Main lesson
matplotlib__ matplotlib-24570	Live-SWE Full resolves, but Live-SWE without the runtime skill, the shared baseline, and TRAE Full all fail.	A moderate delegated topology can help on some boundary cases, but the same sample also shows that a much heavier candidate-search topology can overshoot the task.
django__ django-14404	Live-SWE Full fails, while lighter coding conditions resolve the task.	Some local repository repairs favor the shortest direct patch path, so extra workflow structure can add friction instead of robustness.
sympy__ sympy-23950	The heavier Full conditions fail, while lighter structured conditions resolve.	Component value is task-dependent: additional orchestration is not uniformly helpful when the bug mainly requires a tight local repair loop.
django__ django-13406	Structured runs report successful local revalidation, yet the official evaluator still fails.	Local acceptance layers can diverge from the benchmark’s final acceptance object, so extra verification structure is only useful when it stays aligned with evaluator behavior.

- **Minimal delegated baseline.** If no harness skill is loaded, or if the loaded skills are incomplete, the runtime first constructs the thinnest runnable baseline from the benchmark contract and then treats extra skills as overlays on that baseline. This is the shared substrate behind the RQ1 “w/o harness skill” condition.
- **Call-graph recovery with explicit context semantics.** The runtime reconstructs roles, stages, repetition structure, and independence requirements from the skill text, and then realizes them as child-agent launches. `fork_context=true` means that a child forks and inherits the parent’s accumulated conversational context. `fork_context=false` means that a child starts from a fresh, independent, clean context and receives only the minimal task packet explicitly handed to it. Together with disposable one-shot children and fresh children for independent branches, this preserves the original harness’s model-call boundaries instead of collapsing everything into one long dialogue.
- **Separated runtime state and final artifacts.** Durable intermediate state is written under `STATE_ROOT` (default `/sa-output/runtime`) only when needed for reuse or auditability, while judgeable deliverables go to `/sa-output/artifacts`. This lets the runtime expose stable evidence surfaces without mirroring the entire task workspace.
- **Contract-first completion and auditability.** Benchmark outputs and completion gates remain the primary contract, but the runtime must leave inspectable evidence when a harness claims staged or multi-role execution. As a result, removing the runtime skill in RQ1 removes a shared layer of orchestration, context, artifact, and reporting discipline rather than merely deleting extra prompt text.

D Supplementary RQ1 case notes

Table 7 lists representative SWE cases that shape our RQ1 interpretation. The goal is not exhaustive error taxonomy. Instead, the table isolates a few component-sensitive samples that expose the main mechanisms behind the paired flips: moderate structure helping, over-expanded search hurting, direct-path over-structuring, and local-verifier mismatch.

E Supplementary RQ2 analysis

Figure 4 adds two complementary views for SWE RQ2. Estimated API cost uses the public GPT-5.4 text-token rates on OpenAI’s API pricing page as accessed on March 26, 2026 ($2.50/\text{M input}$ and $15.00/\text{M output}$).¹ Because our logs only expose aggregate prompt and completion totals, we exclude cached-input discounts, context-length surcharges above 270K, and tool or container fees.

¹<https://openai.com/api/pricing/>

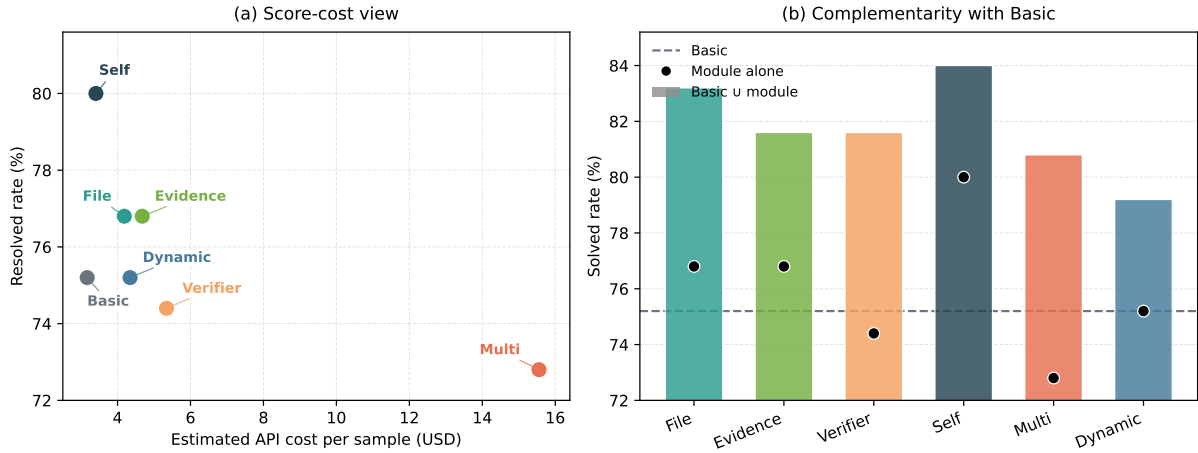


Figure 4: **Supplementary views for SWE RQ2.** Left: resolved rate versus estimated token-based API cost per sample under public GPT-5.4 text pricing. Right: standalone solved rate and union solved rate with Basic for each ablated module.

The left panel separates the modules more clearly than the score table alone. Self-evolution is the only module that moves upward without moving far right, which matches the claim that it tightens the solve loop rather than simply buying a larger search tree. File-backed state and evidence-backed answering move moderately right for only mild score gains, which is consistent with process-structure benefits rather than large correctness gains. Verifier and especially multi-candidate search are dominated in this view, while dynamic orchestration stays near Basic in score but not in cost.

The right panel explains why some score-neutral or slightly negative modules are still behaviorally interesting. Dynamic orchestration and verifier still enlarge the Basic union solved set even when their standalone score is weak, so they change which boundary cases are recoverable rather than merely leaving behavior unchanged.

Self-evolution positive case: `scikit-learn__scikit-learn-25747`. Basic fails this sample, but self-evolution resolves it. The trajectory organizes the run around an explicit attempt contract in which Attempt 1 is treated as successful only if the task acceptance gate is satisfied. In this case, the system closes the run after Attempt 1 rather than expanding into a larger retry tree, and the evaluator confirms that the final patch fixes the target FAIL_TO_PASS tests. This is the favorable regime for self-evolution: the extra structure makes the first repair attempt more disciplined and better aligned with the benchmark gate.

File/evidence positive case: `mwaskom__seaborn-3069`. Basic fails this sample, while both file-backed state and evidence-backed answering resolve it. Under file-backed state, the workspace leaves a durable spine consisting of a parent response, append-only task history, and manifest entries for the promoted patch artifact, which makes the child handoff and artifact lineage explicit. Under evidence-backed answering, the run produces a standalone analysis artifact that ties the patch to direct observations, root-cause reasoning, and focused validation on the nominal-axis regressions. Taken together, the pair shows that these modules are strongest when cleaner state handoff and release discipline help the solver keep one patch surface and one verification story.

Verifier positive case: `django__django-11734`. Verifier helps when the central claim can be checked independently and narrowly. In this sample, the verifier stage does more than restate the patch: it reruns targeted Django tests around OuterRef behavior, checks the resulting correlated-query behavior, and inspects whether the generated SQL binds against the outer-model columns that define the bug. The benchmark then agrees with that judgment and marks the sample resolved. This is the regime in which verifier adds value: the verifier’s local acceptance object is close to the benchmark’s final acceptance gate.

Shared counterexample: `sympy__sympy-23950`. This sample is resolved by Basic and self-evolution, but file-backed state, evidence-backed answering, verifier, dynamic orchestration, and multi-candidate

search all fail it. The verifier run is especially informative because the final response explicitly says that a separate verifier reported “solved,” while the official evaluator still fails `test_as_set`. This is a compact example of the broader RQ2 warning sign: extra process layers can make a run more structured and locally convincing while still drifting away from the benchmark’s actual acceptance object. That is why RQ2 is better read as a study of alignment between intermediate control structure and final evaluator behavior, not as a monotonic story about adding more structure.

F Shared modules used in RQ2

The module boxes below are concise paraphrastic summaries of the shared behaviors used in RQ2 rather than verbatim copies of the original skill text.

file-backed state

ROOT: Choose `STATE_ROOT` under `/sa-output`, keep it separate from the original task workspace, and maintain `STATE_ROOT/RESPONSE.md` as the stable runtime-level status file.
 HANDOFF: No prompt, role instruction, reply, or promoted artifact counts as transferred until it exists as `TASK.md`, `SKILL.md`, `RESPONSE.md`, or another named file under `STATE_ROOT`.
 CHILD PACKET: Each launched child receives `children/<id>/TASK.md`, optional `children/<id>/SKILL.md`, and writes back `children/<id>/RESPONSE.md`.
 BOOKKEEPING: Keep append-only launch and promotion history in `state/task_history.jsonl`, index promoted outputs in `artifacts/manifest.json`, and reopen files by path for reuse and recovery.

evidence-backed answering

ARTIFACT: Before any final answer, final patch, or solved claim, write one standalone evidence document as the designated evidence artifact for the current task or stage.
 STRUCTURE: Cover the problem statement, relevant materials, observed symptoms, root cause, candidate resolution, validation, and residual uncertainty.
 CLAIM DISCIPLINE: Each major claim must state its provenance, whether it is direct observation or inference, and the minimal supporting span or output segment when available.
 GATE: Do not release a complete answer while release-critical claims remain uncited, contradicted, or materially incomplete in that evidence document.

verifier separation

ROLE: Verifier inspects one candidate answer against the original problem and the lightest sufficient task materials needed to check it.
 PROCEDURE: Identify the candidate’s claim, break it into checkable subclaims, audit completeness, factual correctness, and logical correctness, and run at least one central independent check when feasible.
 OUTPUT: Return exactly one primary verdict label plus a report that explains the verdict, names the checks run or blocked, and does not repair the candidate on its behalf.

self-evolution

LOOP: Run an explicit retry loop with a real baseline attempt first and a default cap of five attempts unless the task specifies otherwise.
 TRIGGER: After every non-successful, partially successful, unstable, or stalled attempt, reflect on concrete failure signals before planning the next attempt.
 AXES: Redesign the next attempt along prompt, tool, and workflow evolution, and make attempt 2 materially reflect the reflection from attempt 1.
 STOP: Continue until judged success or the attempt cap is reached, and report incomplete rather than pretending the last attempt passed.

multi-candidate search

BUDGET: Use an explicit candidate budget `K`, defaulting to `K=5` when unspecified, and restore lost budget if a branch crashes before returning comparable evidence.
 DIVERSITY: Vary the core hypothesis, decomposition, evidence route, tool plan, or risk preference so candidates are not near-duplicates.
 SELECTION: Prune duplicates, unsupported, dominated, or overly risky branches, then compare survivors on task fit, evidence quality, coherence, and repair cost.
 ESCALATION: If no candidate is good enough, expand or redesign the search instead of forcing a fragile winner.

dynamic orchestration

AUTONOMY: Beyond the mandatory task-owning child, add extra subagents only when delegation materially improves coverage, latency, specialist focus, or quality control, and prefer the smallest adequate topology.

TOPOLOGY: Classify the task shape, assign each child a non-overlapping responsibility and success condition, and parallelize only genuinely independent branches.

PARENT ROLE: Once a delegated topology is chosen, the parent should narrate launches, waits, comparisons, and integration rather than child-owned substantive actions.

BOUNDARY: Direct task-workspace familiarization or repository probing belongs to child roles after commitment to delegated execution, not to the parent.